

RÉPUBLIQUE TUNISIENNE ★★★ MINISTÈRE DE L'ÉDUCATION	EXAMEN DU BACCALAURÉAT SESSION 2022		NOUVEAU RÉGIME
	ÉPREUVE PRATIQUE D'INFORMATIQUE		
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques		
	Coefficient de l'épreuve : 0.5		Durée : 1h

Important :

1. Une solution modulaire au problème posé est exigée.
2. Dans le répertoire **Bac2022**, créez un dossier de travail ayant comme nom votre numéro d'inscription (6 chiffres) et dans lequel vous devez enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Le nombre ondulant

Un nombre **N** est dit **ondulant** s'il est formé de trois chiffres au minimum et qui est de la forme **ababab...** avec **a ≠ b**.

Exemples :

- 101, 2525, 56565 et 1717171 sont des nombres ondulants.
- 12345 et 808008 ne sont pas des nombres ondulants

Pour vérifier si un entier naturel **N** ($N \geq 100$) est un nombre **ondulant** ou non, on se propose de concevoir une interface graphique contenant les éléments suivants :

- Un label contenant le texte : "**Nombre ondulant** "
- Un label demandant la saisie d'un nombre "**Introduire un entier ≥ 100 :** "
- Une zone de saisie permettant la saisie du nombre
- Un bouton intitulé "**Vérifier**"
- Un label pour afficher le message adéquat

Nombre ondulant

Introduire un entier ≥ 100 :

Travail demandé :

- 1) Concevoir une interface graphique comme illustrée ci-dessus et l'enregistrer, dans votre dossier de travail, sous le nom "**InterfaceOndulant**".
- 2) Créer un programme Python et l'enregistrer, dans votre dossier de travail, sous le nom "**NbrOndulant**".
- 3) Développer, dans le programme "**NbrOndulant**", une fonction **Ondulant(N)** qui permet de vérifier si un entier **N** est ondulant ou non.
- 4) Dans le programme "**NbrOndulant**" :
 - ajouter les instructions permettant d'appeler l'interface graphique intitulée "**InterfaceOndulant**" en exploitant l'annexe ci-après.
 - développer un module "**Play**", qui s'exécute suite à un clic sur le bouton "**Vérifier**", permettant de récupérer l'entier **N** saisi, puis d'exploiter la fonction "**Ondulant**" afin d'afficher le message adéquat via le **label** dédié à l'affichage de l'interface "**InterfaceOndulant**".

N.B. : l'affichage du message doit être conforme aux exemples d'exécution suivants :

Exemples d'exécution :

Nombre ondulant Introduire un entier ≥ 100 : 27 Vérifier Veuillez introduire un nombre ≥ 100	Nombre ondulant Introduire un entier ≥ 100 : 56565 Vérifier 56565 est ondulant
Nombre ondulant Introduire un entier ≥ 100 : 808008 Vérifier 808008 n'est pas ondulant	

Annexe

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi ("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect (Nom_Module)
app.exec_()
```

Grille d'évaluation

Tâches	Nombre de points
Conception de l'interface "InterfaceOndulant"	4 pts
Création et enregistrement du programme "NbrOndulant"	1 pt
Développement de la fonction "Ondulant"	6 pts
Ajout des instructions : - de l'interface "InterfaceOndulant" - du module "Play"	2 pts 4 pts
Importation des bibliothèques nécessaires, modularité et cohérence	3 pts

CORRECTION BAC TP 2022 8H ondulant

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication

def ondulant(x):
    ch=str(x)
    test=ch[0]!=ch[1]
    i=2
    while test and i<=len(ch)-1:
        if ch[i]==ch[i-2]:
            i=i+1
        else:
            test=False
    return test

def ondulant2(x):          # deuxieme méthode
    ch=str(x)
    test=ch[0]!=ch[1]
    i=2
    while test and i<=len(ch)-1:
        test=ch[i]==ch[i-2] #ou test=ch[i]==ch[i%2]
        i+=1
    return test

def play():
    ch=f.t.text()
    if not(ch.isdecimal()) or (int(ch)<100):
        msg="veuillez introduire un nombre >=100"
    elif ondulant(int(ch)):
        msg=ch+" est un nombre ondulant"
    else:
        msg=ch+" n'est pas un nombre ondulant"
    f.l.setText(msg)

app=QApplication([])
f=loadUi("ondulant.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()
```

Liste des nombres ondulants

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication

def ondulant(x):
    ch=str(x)
    test=ch[0]!=ch[1]
    i=2
    while test and i<=len(ch)-1:
        if ch[i]==ch[i-2]:
            i=i+1
        else:
            test=False
    return test

def nbrondulant(n,m):
    ch=""
    for i in range(n,m+1):
        if ondulant(i):
            ch=ch+str(i)+'|'
    return ch

def play():
    ch1=f.t1.text()
    ch2=f.t2.text()
    if not(ch1.isdecimal()) or (int(ch1)<100) or not(ch2.isdecimal()) or (int(ch2)<100):
        msg="veuillez introduire deux nombres >=100"
    elif nbrondulant(int(ch1),int(ch2))=="":
        msg="aucun nombre ondulant entre : "+ch1+' '+ch2
    else:
        msg="le nombres ondulant entre "+ch1+" "+ch2+"= "+nbrondulant(int(ch1),int(ch2))
    f.l.setText(msg)

app=QApplication([])
f=loadUi("ondulant.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()
```

RÉPUBLIQUE TUNISIENNE *** MINISTÈRE DE L'ÉDUCATION	EXAMEN DU BACCALAURÉAT SESSION 2022		NOUVEAU RÉGIME
	ÉPREUVE PRATIQUE D'INFORMATIQUE		
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques		
	Coefficient de l'épreuve : 0.5		Durée : 1h

Important :

1. Une solution modulaire au problème posé est exigée.
2. Dans le répertoire **Bac2022**, créez un dossier de travail ayant comme nom votre numéro d'inscription (6 chiffres) et dans lequel vous devez enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Le nombre super-pairplus

Un nombre N est dit **super-pairplus** s'il vérifie les trois conditions suivantes :

- **Condition 1 :** N est pair
- **Condition 2 :** N est formé uniquement par des chiffres pairs
- **Condition 3 :** tous les diviseurs de N autre que 1 sont des entiers pairs.

Exemples :

- $N = 64$ est un entier **super-pairplus** car 64 vérifie les trois conditions :
 - **Condition 1 :** 64 est pair
 - **Condition 2 :** 64 est formé uniquement par des chiffres pairs (6 et 4)
 - **Condition 3 :** les diviseurs de 64 autre que 1 (2, 4, 8, 16, 32 et 64) sont des entiers pairs.
- $N = 28$ n'est pas un entier **super-pairplus** car une des trois conditions (**condition 3**) n'est pas vérifiée :
 - **Condition 1 :** 28 est pair
 - **Condition 2 :** 28 est formé uniquement par des chiffres pairs (2 et 8)
 - **Condition 3 :** les diviseurs de 28 autre que 1 (2, 4, 7, 14 et 28) ne sont pas tous des entiers pairs. En effet, 7 est impair

Pour vérifier si un entier naturel N ($N > 0$) est un nombre **super-pairplus** ou non, on se propose de concevoir une interface graphique contenant les éléments suivants :

- Un label contenant le texte : "**Nombre super-pairplus**"
- Un label demandant la saisie d'un nombre "**Introduire un entier > 0 :**"
- Une zone de saisie permettant la saisie du nombre
- Un bouton intitulé "**Vérifier**"
- Un label pour afficher le message adéquat

Nombre super-pairplus

Introduire un entier > 0 :

Travail demandé :

- 1) Concevoir une interface graphique comme illustrée ci-dessus et l'enregistrer, dans votre dossier de travail, sous le nom "**InterfaceSuperPairplus**".
- 2) Créer un programme Python et l'enregistrer, dans votre dossier de travail, sous le nom "**NbrSuperPairplus**".
- 3) Développer, dans le programme "**NbrSuperPairplus**", une fonction **SuperPairplus (N)** qui permet de vérifier si un entier N est super-pairplus ou non.

4) Dans le programme "NbrSuperPairplus" :

- ajouter les instructions permettant d'appeler l'interface graphique intitulée "InterfaceSuperPairplus" en exploitant l'annexe ci-après.
- développer un module "Play", qui s'exécute suite à un clic sur le bouton "Vérifier", permettant de récupérer l'entier N saisi, puis d'exploiter la fonction "SuperPairplus" afin d'afficher le message adéquat via le label dédié à l'affichage de l'interface "InterfaceSuperPairplus".

N.B. : l'affichage du message doit être conforme aux exemples d'exécution suivants :

Exemples d'exécution :

Nombre super-pairplus
 Introduire un entier > 0 :

 Veuillez introduire un nombre > 0

Nombre super-pairplus
 Introduire un entier > 0 :

 64 est super-pairplus

Nombre super-pairplus
 Introduire un entier > 0 :

 28 n'est pas super-pairplus

Annexe

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect(Nom_Module)
app.exec_()
```

Grille d'évaluation

Tâches	Nombre de points
Conception de l'interface "InterfaceSuperPairplus"	4 pts
Création et enregistrement du programme "NbrSuperPairplus"	1 pt
Développement de la fonction "SuperPairplus"	6 pts
Ajout des instructions : ▪ de l'interface "InterfaceSuperPairplus" ▪ du module "Play"	2 pts 4 pts
Importation des bibliothèques nécessaires, modularité et cohérence	3 pts

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication

def super_pairplus(x):
    test=pairplus(x)
    i=2
    while test and i<=x:
        if x%i==0:
            test=i%2==0
            i=i+1
    return test

def pairplus(x):
    test=x%2==0
    while test==True and x!=0:
        if x%2!=0:
            test=False
        else:
            x=x//10
    return test

def play():
    ch=f.t.text()
    if not(ch.isdecimal()) or (int(ch)<=0):
        msg="veuillez introduire un nombre >0"
    elif super_pairplus(int(ch)):
        msg=ch+" est un nombre super pair plus"
    else:
        msg=ch+" n'est pas un nombre super_pairplus"
    f.l.setText(msg)
app=QApplication([])
f=loadUi("super_pairplus.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()
```

RÉPUBLIQUE TUNISIENNE *** MINISTÈRE DE L'ÉDUCATION	EXAMEN DU BACCALAURÉAT SESSION 2022		NOUVEAU RÉGIME
	ÉPREUVE PRATIQUE D'INFORMATIQUE		
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques		
	Coefficient de l'épreuve : 0.5		Durée : 1h

Important :

1. Une solution modulaire au problème posé est exigée.
2. Dans le répertoire **Bac2022**, créez un dossier de travail ayant comme nom votre numéro d'inscription (6 chiffres) et dans lequel vous devez enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Le nombre semi-premier

Un nombre N est dit **semi-premier** lorsqu'il est égal au produit de deux nombres premiers non nécessairement distincts. C'est-à-dire $N = k \cdot k$ avec k est un nombre premier ou $N = k \cdot j$ avec k et j sont deux nombres premiers.

Exemples :

- 6 est un nombre semi-premier car $6 = 2 \times 3$ avec 2 et 3 sont deux nombres premiers.
- 25 est un nombre semi-premier car $25 = 5 \times 5$ avec 5 est un nombre premier.
- 831 est un nombre semi-premier car $831 = 3 \times 277$ avec 3 et 277 sont deux nombres premiers
- 8 n'est pas un nombre semi-premier, car $8 = 2 \times 4$ avec 4 n'est pas un nombre premier.

Pour vérifier si un entier naturel N ($N > 2$) est un nombre **semi-premier** ou non, on se propose de concevoir une interface graphique contenant les éléments suivants :

- Un label contenant le texte : "Nombre semi-premier"
- Un label demandant la saisie d'un nombre : "Introduire un entier > 2 : "
- Une zone de saisie permettant la saisie du nombre
- Un bouton intitulé "Vérifier"
- Un label pour afficher le message adéquat

Nombre semi-premier

Introduire un entier > 2 :

Travail demandé :

- 1) Concevoir une interface graphique comme illustrée ci-dessus et l'enregistrer, dans votre dossier de travail, sous le nom "InterfaceSemiPremier".
- 2) Créer un programme Python et l'enregistrer, dans votre dossier de travail, sous le nom "NbrSemiPremier".
- 3) Développer, dans le programme "NbrSemiPremier", une fonction **SemiPremier(N)** qui permet de vérifier si un entier N est semi-premier ou non.

4) Dans le programme "NbrSemiPremier" :

- ajouter les instructions permettant d'appeler l'interface graphique intitulée "InterfaceSemiPremier" en exploitant l'annexe ci-après.
- développer un module "Play", qui s'exécute suite à un clic sur le bouton "Vérifier", permettant de récupérer l'entier N saisi, puis d'exploiter la fonction "SemiPremier" afin d'afficher le message adéquat via le **label** dédié à l'affichage de l'interface "InterfaceSemiPremier".

N.B. :

- Le candidat est appelé à développer un module qui permet de vérifier la primalité d'un entier **sans faire recours** à des fonctions prédéfinies telles que `isprime()`.
- l'affichage du message doit être conforme aux exemples d'exécution suivants :

Exemples d'exécution :

Nombre semi-premier
Introduire un entier > 2 :

Veuillez introduire un nombre > 2

Nombre semi-premier
Introduire un entier > 2 :

831 est semi-premier

Nombre semi-premier
Introduire un entier > 2 :

8 n'est pas semi-premier

Annexe

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi ("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect (Nom_Module)
app.exec_()
```

Grille d'évaluation

Tâches	Nombre de points
Conception de l'interface "InterfaceSemiPremier"	4 pts
Création et enregistrement du programme "NbrSemiPremier"	1 pt
Développement de la fonction "SemiPremier"	6 pts
Ajout des instructions :	2 pts
▪ de l'interface "InterfaceSemiPremier"	4 pts
▪ du module "Play"	
Importation des bibliothèques nécessaires, modularité et cohérence	3 pts

BAC 2022 11H semi premier

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication

def premier(n):
    test=True
    i=2
    while test and i<=n//2:
        if n%i==0:
            test=False
        else:
            i=i+1
    return test

def SemiPremier(x):
    test=False
    i=2
    while not test and i<=x//2:
        if x%i==0 and premier(i) and premier(x//i):
            test=True
        else:
            i=i+1
    return test

def play():
    ch=f.t.text()
    if not(ch.isdecimal()) or (int(ch)<=2):
        msg="veuillez introduire un nombre >2"
    elif SemiPremier(int(ch)):
        msg=ch+" est un nombre semi premier"
    else:
        msg=ch+" n'est pas un nombre semi premier"
    f.l.setText(msg)

app=QApplication([])
f=loadUi("InterfaceSemiPremier.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()
```

RÉPUBLIQUE TUNISIENNE ★ ★ ★ MINISTÈRE DE L'ÉDUCATION	EXAMEN DU BACCALAURÉAT SESSION 2022		NOUVEAU RÉGIME
	ÉPREUVE PRATIQUE D'INFORMATIQUE		
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques		
	Coefficient de l'épreuve : 0.5		Durée : 1h

Important :

1. Une solution modulaire au problème posé est exigée.
2. Dans le répertoire **Bac2022**, créez un dossier de travail ayant comme nom votre numéro d'inscription (6 chiffres) et dans lequel vous devez enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Le nombre lisse

Un nombre **N** est dit **lisse** lorsque son **plus grand diviseur premier** est inférieur ou égal à la racine carrée du nombre **N**.

Exemples :

- **N = 60**, les diviseurs de 60 sont : 1, 2, 3, 4, 5, 6, 10, 12, 15, 30 et 60. Son plus grand diviseur premier est 5 et puisque $5 \leq \sqrt{60} = 7.746 \dots$, donc **60 est un nombre lisse**.
- **N = 49**, les diviseurs de 49 sont : 1, 7 et 49. Son plus grand diviseur premier est 7 et puisque $7 \leq \sqrt{49} = 7$ donc **49 est un nombre lisse**.
- **N = 22**, les diviseurs de 22 sont : 1, 2, 11 et 22. Son plus grand diviseur premier est 11 et puisque $11 > \sqrt{22} = 4.690 \dots$, donc **22 n'est pas un nombre lisse**.

Pour vérifier si un entier naturel **N** ($N > 1$) est un nombre **lisse** ou non, on se propose de concevoir une interface graphique contenant les éléments suivants :

- Un label contenant le texte : "**Nombre lisse**"
- Un label demandant la saisie d'un nombre : "**Introduire un entier > 1 :** "
- Une zone de saisie permettant la saisie du nombre
- Un bouton nommé "**Vérifier**"
- Un label pour afficher le message adéquat

Nombre lisse

Introduire un entier > 1 :

Travail demandé :

- 1) Concevoir une interface graphique comme illustrée ci-dessus et l'enregistrer, dans votre dossier de travail, sous le nom "**InterfaceLisse**".
- 2) Créer un programme Python et l'enregistrer, dans votre dossier de travail, sous le nom "**NbrLisse**".
- 3) Développer, dans le programme "**NbrLisse**", une fonction **Lisse(N)** qui permet de vérifier si un entier **N** est lisse ou non.

4) Dans le programme "NbrLisse" :

- ajouter les instructions permettant d'appeler l'interface graphique intitulée "InterfaceLisse" en exploitant l'annexe ci-après.
- développer un module "Play", qui s'exécute suite à un clic sur le bouton "Vérifier", permettant de récupérer l'entier N saisi, puis d'exploiter la fonction "Lisse" afin d'afficher le message adéquat via le label dédié à l'affichage de l'interface "InterfaceLisse".

N.B. :

- Le candidat est appelé à développer un module qui permet de vérifier la primalité d'un entier **sans faire recours** à des fonctions prédéfinies telles que `isprime()`.
- l'affichage du message doit être conforme aux exemples d'exécution suivants :

Exemples d'exécution :

Nombre lisse
Introduire un entier > 1 :

Veuillez introduire un nombre > 1

Nombre lisse
Introduire un entier > 1 :

60 est un nombre lisse

Nombre lisse
Introduire un entier > 1 :

22 n'est pas un nombre lisse

Annexe

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi ("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect (Nom_Module)
app.exec_()
```

Grille d'évaluation

Tâches	Nombre de points
Conception de l'interface "InterfaceLisse"	4 pts
Création et enregistrement du programme "NbrLisse"	1 pt
Développement de la fonction "Lisse"	6 pts
Ajout des instructions :	
▪ de l'interface "InterfaceLisse"	2 pts
▪ du module "Play"	4 pts
Importation des bibliothèques nécessaires, modularité et cohérence	3 pts

bac 2022-13h nombre lisse	Afficher les entiers lisses entre a et b
<pre> from PyQt5.uic import loadUi from PyQt5.QtWidgets import QApplication from math import* def premier(n): test=True i=2 while test and i<=n//2: if n%i==0: test=False else: i=i+1 return test def lisse(x): m=1 for i in range(2,x+1): if x%i==0 and premier(i)and m<i: m=i return m<=sqrt(x) def play(): ch=f.t.text() if not(ch.isdecimal()) or (int(ch)<=2): msg="veuillez introduire un nombre >1" elif lisse(int(ch)): msg=ch+" est un nombre lisse" else: msg=ch+" n'est pas un nombre lisse" f.l.setText(msg) app=QApplication([]) f=loadUi("InterfaceLisse.ui") f.show() f.b.clicked.connect(play) app.exec_() </pre>	<pre> from PyQt5.uic import loadUi from PyQt5.QtWidgets import QApplication from math import* def premier(n): test=True i=2 while test and i<=n//2: if n%i==0: test=False else: i=i+1 return test def lisse(x): m=1 for i in range(2,x+1): if x%i==0 and premier(i)and m<i: m=i return m<=sqrt(x) def nbrlisse(a,b): ch="" for i in range(a,b+1): if lisse(i): ch=ch+" "+str(i) return ch def play(): ch1=f.t1.text() ch2=f.t2.text() if not(ch1.isdecimal()) or (int(ch1)<=2) or not(ch2.isdecimal()) or (int(ch2)<=2): msg="veuillez introduire des nombres >1" else: msg="les nombres lisses entre "+ch1+" et "+ch2+nbrlisse(int(ch1),int(ch2)) f.l.setText(msg) app=QApplication([]) f=loadUi("InterfaceLisse.ui") f.show() f.b.clicked.connect(play) app.exec_() </pre>

RÉPUBLIQUE TUNISIENNE ○○○ Ministère de l'Éducation	EXAMEN DU BACCALAURÉAT	SESSION 2023
	ÉPREUVE PRATIQUE D'INFORMATIQUE	
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques	
	Coefficient de l'épreuve : 0.5	Durée : 1h

Important :

1. Une solution modulaire au problème posé est exigée.
2. Dans le répertoire **Bac2023**, créez un dossier de travail ayant comme nom votre numéro d'inscription (6 chiffres) et dans lequel vous devez enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Succession parfaite

On se propose de concevoir une interface graphique permettant de saisir deux nombres positifs **M** et **N** puis de vérifier s'ils forment une **succession parfaite** ou non.

Une succession parfaite de deux nombres positifs **M** et **N** est une chaîne de caractères **ch** formée par une succession de chiffres consécutifs distincts où le **pas** de la succession est égal à 1. Cette chaîne est obtenue en concaténant les chiffres de **M** et **N** puis en les triant dans l'ordre croissant.

Exemples :

- Pour $M=2748$ et $N=365$, **ch** = "2345678". Les chiffres de **ch** forment une **succession parfaite**. En effet, le **pas** de la succession est égal à 1 entre tous les chiffres de **ch**.
- Pour $M=8473$ et $N=546$, **ch** = "3445678". Les chiffres de **ch** ne forment pas une **succession parfaite**. En effet, le **pas** de la succession est différent de 1 entre le deuxième et le troisième chiffre.
- Pour $M=2748$ et $N=956$, **ch** = "2456789". Les chiffres de **ch** ne forment pas une **succession parfaite**. En effet, le **pas** de la succession est différent de 1 entre le premier et le deuxième chiffre.

L'interface graphique à concevoir contient les éléments suivants, comme l'illustre la capture d'écran ci-dessous :

- Un label contenant le texte " **Succession parfaite** "
- Un label contenant le texte " **M** = "
- Une zone de saisie pour la saisie du nombre **M**
- Un label contenant le texte " **N** = "
- Une zone de saisie pour la saisie du nombre **N**
- Un label pour afficher le message adéquat
- Un bouton intitulé " **Vérifier** "

Travail demandé :

- 1) Concevoir l'interface graphique présentée précédemment et l'enregistrer sous le nom **InterfaceSuccession**.
- 2) Créer un programme Python et l'enregistrer sous le nom **Succession**, dans lequel, il est demandé :
 - a) de développer une fonction nommée **Verifier (M,N)** qui permet de vérifier si **M** et **N** forment une succession parfaite ou non.

- b) de développer un module **Play** qui s'exécute suite à un clic sur le bouton "**Vérifier**", permettant :
- de récupérer les deux entiers **M** et **N** saisis qui doivent être positifs.
 - d'exploiter la fonction **Verifier (M,N)** afin d'afficher le message adéquat via le **label** dédié à l'affichage dans l'interface graphique **InterfaceSuccession**.
- c) d'ajouter les instructions permettant d'exploiter l'interface graphique intitulée **InterfaceSuccession** en se référant à l'annexe ci-après.

N.B. : l'affichage doit être conforme aux exemples d'exécutions suivants :

Exemples d'exécutions :

Succession parfaite

M =

N =

Veuillez introduire 2 entiers positifs

Vérifier

Succession parfaite

M =

N =

2748 et 365 forment une succession parfaite

Vérifier

Succession parfaite

M =

N =

8473 et 546 ne forment pas une succession parfaite

Vérifier

Annexe

```

from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi ("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect (Nom_Module)
app.exec_()
```

Grille d'évaluation

Tâches	Nombre de points
Conception de l'interface InterfaceSuccession	4 pts
Création et enregistrement du programme Succession	1 pt
Développement de la fonction Verifier	6 pts
Développement du module Play	4 pts
Ajout des instructions de l'exploitation de l'interface	3 pts
Modularité et cohérence	2 pts

bac 2023 8h Succession parfait

```
from PyQt5.QtWidgets import *
from PyQt5.uic import *
def tri(ch):
    for i in range(len(ch)-1):
        mx=i
        for j in range(i+1,len(ch)):
            if ch[j]<ch[mx]:
                mx=j
        if mx!=i:
            ch[ch[:i]+ch[mx]+ch[i:mx]+ch[mx+1:]]
    return ch
def succesion(ch):
    test=True
    i=0
    while i<len(ch)-2 and test:
        if int(ch[i+1])-int(ch[i])==1:
            i=i+1
        else:
            test=False
    return test
def play():
    ch1=f.t1.text()
    ch2=f.t2.text()
    ch=ch1+ch2
    if not(ch1.isdecimal()) or not(ch2.isdecimal()) or not(int(ch1)>=0) or not(int(ch2)>=0):
        msg="veuillez introduire deux entiers positifs"
    elif succesion(tri(ch)):
        msg=ch1+' et '+ch2+' forment une succession parfaite'
    else:
        msg=ch1+' et '+ch2+' ne forment pas une succession parfaite'
    f.l.setText(msg)
app=QApplication([])
f=loadUi("InterfaceSuccession.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()
```

RÉPUBLIQUE TUNISIENNE *** Ministère de l'Éducation	EXAMEN DU BACCALAURÉAT	SESSION 2023
	ÉPREUVE PRATIQUE D'INFORMATIQUE	
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques	
	Coefficient de l'épreuve : 0.5	Durée : 1h

Important :

1. Une solution modulaire au problème posé est exigée.
2. Dans le répertoire **Bac2023**, créez un dossier de travail ayant comme nom votre numéro d'inscription (6 chiffres) et dans lequel vous devez enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Rotation - Miroir

On se propose de concevoir une interface graphique permettant de saisir une chaîne de caractères **ch** et de la crypter en utilisant le principe suivant :

➤ **Etape 1 : Rotation13**

Effectuer, pour chaque caractère de **ch**, une rotation de **13** caractères dans l'ordre alphabétique en appliquant la formule $\text{Chr}(97 + (\text{Ord}(\text{ch}[i]) - 97 + 13) \text{ Mod } 26)$ pour obtenir le caractère correspondant.

➤ **Etape 2 : Miroir**

Transformer la chaîne de caractères obtenue à l'issue de l'**Etape 1** en son miroir. C'est-à-dire permuter le premier caractère avec le dernier, le deuxième caractère avec l'avant dernier et ainsi de suite.

Exemple :

Pour **ch** = "algo"

➤ **Etape 1 : Rotation13**

- le caractère "a" sera crypté en "n". En effet, $\text{Chr}(97 + (\text{Ord}(\text{"a"}) - 97 + 13) \text{ Mod } 26) = \text{Chr}(110)$ qui correspond au caractère "n"
- le caractère "l" sera crypté en "y". En effet, $\text{Chr}(97 + (\text{Ord}(\text{"l"}) - 97 + 13) \text{ Mod } 26) = \text{Chr}(121)$ qui correspond au caractère "y".
- le caractère "g" sera crypté en "t". En effet, $\text{Chr}(97 + (\text{Ord}(\text{"g"}) - 97 + 13) \text{ Mod } 26) = \text{Chr}(116)$ qui correspond au caractère "t".
- le caractère "o" sera crypté en "b". En effet, $\text{Chr}(97 + (\text{Ord}(\text{"o"}) - 97 + 13) \text{ Mod } 26) = \text{Chr}(98)$ qui correspond au caractère "b".

D'où le résultat de l'étape 1 est "nytb"

➤ **Etape 2 : Miroir**

Le miroir de la chaîne "nytb" est "btyn"

Donc, le cryptage de la chaîne "algo" en appliquant le principe ci-dessus est "btyn"

L'interface graphique à concevoir contient les éléments suivants, comme l'illustre la capture d'écran ci-dessous :

- Un label contenant le texte "Rotation - Miroir"
- Un label contenant le texte "Introduire une chaîne : "
- Une zone de saisie pour la saisie d'une chaîne
- Un label pour afficher le résultat
- Un bouton intitulé "Crypter"

Rotation - Miroir

Introduire une chaîne :

Travail demandé :

- 1) Concevoir l'interface graphique présentée précédemment et l'enregistrer sous le nom **InterfaceRotationMiroir**.
- 2) Créer un programme Python et l'enregistrer sous le nom **RotationMiroir**, dans lequel, il est demandé :
 - a) d'implémenter l'algorithme suivant de la fonction **Rotation** qui permet de retourner le résultat de l'étape 1.

```
Fonction Rotation (ch : Chaîne de caractères) : Chaîne de caractères  
DEBUT  
res ← ""  
Pour i de 0 à Long(ch)-1 Faire  
    res ← res + Chr(97 + (Ord(ch[i]) – 97 + 13) Mod 26)  
Fin Pour  
Retourner (res)  
FIN
```

Déclaration des objets

Objet	Type/Nature
i	Entier
res	Chaîne de caractères

- b) de développer une fonction nommée **Miroir (ch)** qui permet de retourner le miroir d'une chaîne de caractères **ch**.
- c) de développer un module **Play**, qui s'exécute suite à un clic sur le bouton "**Crypter**", permettant :
 - de récupérer la chaîne **ch** saisie. La chaîne **ch** doit être non vide, de longueur inférieure à 10 et contient seulement des lettres alphabétiques en minuscule.
 - de déterminer la chaîne cryptée en utilisant les deux fonctions **Rotation (ch)** et **Miroir (ch)** et d'afficher le résultat via le **label** dédié à l'affichage dans l'interface graphique **InterfaceRotationMiroir**.
- d) d'ajouter les instructions permettant d'exploiter l'interface graphique intitulée **InterfaceRotationMiroir** en se référant à l'annexe ci-après.

N.B. : l'affichage doit être conforme aux exemples d'exécutions suivants :

Exemples d'exécutions :

Rotation - Miroir
Introduire une chaîne :
Veuillez introduire une chaîne
Crypter

Rotation - Miroir
Introduire une chaîne :
Veuillez introduire une chaîne valide !
Crypter

Rotation - Miroir
Introduire une chaîne :
La chaîne cryptée est : btyn
Crypter

Annexe

```
from PyQt5.uic import loadUi  
from PyQt5.QtWidgets import QApplication  
.....  
.....  
app = QApplication([])  
windows = loadUi ("Nom_Interface.ui")  
windows.show()  
windows.Nom_Bouton.clicked.connect (Nom_Module)  
app.exec_()
```

```

from PyQt5.QtWidgets import *
from PyQt5.uic import *
def inverser(ch):
    r=""
    for i in range(len(ch)):
        r=ch[i]+r
    return r
def miroir(ch):
    res=""
    for i in range(len(ch)):
        res=res+chr(97+(ord(ch[i])-97+13)%26)
    return res
def verif(ch):
    i=0
    test=True
    while test and i<len(ch):
        if 'a'<=ch[i]<='z':
            i=i+1
        else:
            test=False
    return test
def play():
    ch=f.t.text()
    if not(1<=len(ch)<=10):
        msg='Veuillez introduire une chaine'
    elif verif(ch)==False:
        msg='Veuillez introduire une chaine valide !'
    else:
        msg=inverser(miroir(ch))
    f.r.setText(msg)
app=QApplication([])
f=loadUi("InterfaceRotationMiroir.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()

```

RÉPUBLIQUE TUNISIENNE *** Ministère de l'Éducation	EXAMEN DU BACCALAURÉAT	SESSION 2023
	ÉPREUVE PRATIQUE D'INFORMATIQUE	
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques	
	Coefficient de l'épreuve : 0.5	Durée : 1h

Important :

1. Une solution modulaire au problème posé est exigée.
2. Dans le répertoire **Bac2023**, créez un dossier de travail ayant comme nom votre numéro d'inscription (6 chiffres) et dans lequel vous devez enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Miroirs de mots

On se propose de concevoir une interface graphique permettant de saisir une chaîne de caractères **ch** et de la crypter en formant une nouvelle chaîne par les miroirs des mots de **ch** dans leur ordre d'apparition. On rappelle que le miroir d'un mot consiste à permuter le premier caractère avec le dernier, le deuxième caractère avec l'avant dernier et ainsi de suite.

Exemple :

Pour **ch** = "La vie est une aventure merveilleuse"

On obtient la chaîne cryptée : "aL eiv tse enu erutneva esuellievrem"

L'interface graphique à concevoir contient les éléments suivants, comme l'illustre la capture d'écran ci-dessous :

- Un label contenant le texte "Miroirs de mots"
- Un label contenant le texte "Introduire une chaîne : "
- Une zone de saisie pour la saisie d'une chaîne
- Un label pour afficher le résultat
- Un bouton intitulé "Miroir"

Travail demandé :

- 1) Concevoir l'interface graphique présentée précédemment et l'enregistrer sous le nom **InterfaceMiroirsMots**.
- 2) Créer un programme Python et l'enregistrer sous le nom **MiroirsMots**, dans lequel, il est demandé :
 - a) de développer une fonction nommée **Miroir (M)** qui permet de retourner le miroir d'un mot **M**.
 - b) de développer un module **Play**, qui s'exécute suite à un clic sur le bouton "Miroir", permettant :
 - de récupérer la chaîne **ch** saisie. La chaîne **ch** doit être non vide et de longueur inférieure à 50, contient seulement des lettres alphabétiques en minuscule et chaque deux mots consécutifs sont séparés par un seul espace.
 - de déterminer la chaîne cryptée en utilisant la fonction **Miroir (M)** et d'afficher le résultat via le label dédié à l'affichage dans l'interface graphique **InterfaceMiroirsMots**.
 - c) d'ajouter les instructions permettant d'exploiter l'interface graphique intitulée **InterfaceMiroirsMots** en se référant à l'annexe ci-après.

N.B. : l’affichage doit être conforme aux exemples d’executions suivants :

Exemples d’executions :

Miroirs de mots
Introduire une chaîne :
Veuillez introduire une chaîne
Miroir

Miroirs de mots
Introduire une chaîne :
Entre 2 mots un seul espace est autorisé
Miroir

Miroirs de mots
Introduire une chaîne :
aL eiv tse enu erutneva esuellievrem
Miroir

Annexe

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi ("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect (Nom_Module)
app.exec_()
```

Grille d’évaluation

Tâches	Nombre de points
Conception de l’interface " InterfaceMiroirsMots "	4 pts
Création et enregistrement du programme " MiroirsMots "	1 pt
Développement de la fonction " Miroir "	4 pts
Développement du module " Play "	6 pts
Ajout des instructions de l’exploitation de l’interface	3 pts
Modularité et cohérence	2 pts

```

from PyQt5.QtWidgets import *
from PyQt5.uic import *
def miroir(ch):
    res=""
    while ch.find(' ')>=0:
        ch1=ch[:ch.find(' ')]
        ch2=""
        for i in range(len(ch1)):
            ch2=ch1[i]+ch2
        res=res+' '+ch2
        if ch.find(' ')>=0:
            ch=ch[ch.find(' ')+1:]

    for i in range(len(ch)):
        ch2=ch[i]+ch2
    res=res+' '+ch2
    return res
def play():
    ch=f.t.text()
    if ch=="":
        r="Veuillez introduire un phrase"
    elif ch.find(' ')!=-1:
        r="Entre 2 mots un seul espace est autorisé"
    else:
        r=miroir(ch)
    f.l.setText(r)
app=QApplication([])
f=loadUi("InterfaceMiroirsMots.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()

```

RÉPUBLIQUE TUNISIENNE *** Ministère de l'Éducation	EXAMEN DU BACCALAURÉAT	SESSION 2023
	ÉPREUVE PRATIQUE D'INFORMATIQUE	
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques	
	Coefficient de l'épreuve : 0.5	Durée : 1h

Important :

1. Une solution modulaire au problème posé est exigée.
2. Dans le répertoire **Bac2023**, créez un dossier de travail ayant comme nom votre numéro d'inscription (6 chiffres) et dans lequel vous devez enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Triage de mots

On se propose de concevoir une interface graphique permettant de saisir une chaîne de caractères **ch** contenant aux maximum 15 mots puis de trier les mots qui la constituent dans l'ordre croissant de leurs longueurs.

Exemple :

Pour **ch** = "une hirondelle ne fait pas le printemps."

Après le triage des mots de **ch**, on obtient le résultat suivant : "ne le une pas fait printemps hirondelle"

N.B. Le point "." indiquant la fin de la chaîne, ne fait pas partie du dernier mot. Par conséquent ne figure pas dans la chaîne cryptée.

L'interface graphique à concevoir contient les éléments suivants, comme l'illustre la capture d'écran ci-dessous :

- Un label contenant le texte "Triage de mots"
- Un label contenant le texte "Introduire une phrase : "
- Une zone de saisie pour la saisie d'une phrase
- Un label pour afficher le résultat
- Un bouton intitulé "Trier"

Travail demandé :

- 1) Concevoir l'interface graphique présentée précédemment et l'enregistrer sous le nom **InterfaceTriage**.
- 2) Créer un programme Python et l'enregistrer sous le nom **Triage**, dans lequel, il est demandé :
 - a) de développer une fonction nommée **Trier (ch)** qui permet de retourner une chaîne de caractères formée par les mots de la chaîne de caractères **ch** triés dans l'ordre croissant de leurs longueurs.
 - b) de développer un module **Play**, qui s'exécute suite à un clic sur le bouton "Trier", permettant :
 - de récupérer la chaîne **ch** saisie. La chaîne **ch** doit être non vide et de taille inférieure à 50. Elle commence obligatoirement par une lettre, se termine par un point et chaque deux mots consécutifs sont séparés par un seul espace.
 - de trier les mots de la chaîne **ch** en utilisant la fonction **Trier (ch)** puis d'afficher le résultat via le label dédié à l'affichage dans l'interface graphique **InterfaceTriage**.
 - c) d'ajouter les instructions permettant d'exploiter l'interface graphique intitulée **InterfaceTriage** en se référant à l'annexe ci-après.

N.B. : l’affichage doit être conforme aux exemples d’exécutions suivants :

Exemples d’exécutions :

Triage de mots	
Introduire une phrase :	
Veuillez introduire une phrase	
Trier	

Triage de mots	
Introduire une phrase :	bonne chance.
Entre 2 mots un seul espace est autorisé	
Trier	

Triage de mots	
Introduire une phrase :	bonne chance
La chaîne doit se terminer par un point	
Trier	

Triage de mots	
Introduire une phrase :	une hirondelle ne fait pas le printemps.
ne le une pas fait printemps hirondelle	
Trier	

Annexe

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi ("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect (Nom_Module)
app.exec_()
```

Kiteb.net: le site web éducatif

Grille d’évaluation

Tâches	Nombre de points
Conception de l’interface " InterfaceTriage "	4 pts
Création et enregistrement du programme " Triage "	1 pt
Développement de la fonction " Trier "	6 pts
Développement du module " Play "	4 pts
Ajout des instructions de l’exploitation de l’interface	3 pts
Modularité et cohérence	2 pts

```

from PyQt5.QtWidgets import *
from PyQt5.uic import *
from numpy import*
t=array([str]*50)
def trier(ch):
    i=0
    while ch.find(' ')>=0:
        t[i]=ch[:ch.find(' ')]
        if ch.find(' ')>=0:
            i=i+1
            ch=ch[ch.find(' ')+1:]
    t[i]=ch[:len(ch)-1]
    n=i+1
    for i in range(n-1):
        mx=i
        for j in range(i+1,n):
            if len(t[j])<len(t[mx]):
                mx=j
        if mx!=i:
            aux=t[i]
            t[i]=t[mx]
            t[mx]=aux
    rch=""
    for j in range(n):
        rch=rch+t[j]+' '
    return rch
def play():
    ch=f.t.text()
    if not(0<len(ch)<=50) or not('A'<=ch[0].upper()<='Z'):
        msg="Veuillez introduire un phrase "
    elif ch[len(ch)-1]!='.':':
        msg="la chaine doit se termine par un point"
    elif ch.find(' ')!=-1:
        msg="Entre deux mots un seul espace est autorisé"
    else:
        msg=trier(ch)
    f.l.setText(msg)
app=QApplication([])
f=loadUi("InterfaceTriage.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()

```

RÉPUBLIQUE TUNISIENNE *** Ministère de l'Éducation	EXAMEN DU BACCALAURÉAT	SESSION 2023
	ÉPREUVE PRATIQUE D'INFORMATIQUE	
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques	
	Coefficient de l'épreuve : 0.5	Durée : 1h

Important :

1. Une solution modulaire au problème posé est exigée.
2. Dans le répertoire **Bac2023**, créez un dossier de travail ayant comme nom votre numéro d'inscription (6 chiffres) et dans lequel vous devez enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Intersection de deux chaînes

On se propose de concevoir une interface graphique permettant de saisir deux chaînes de caractères **ch1** et **ch2** puis de former la chaîne **res** par les caractères qui apparaissent dans les deux chaînes à la fois.

*N.B. Si un caractère apparaît plusieurs fois dans les deux chaînes, on le mettra une seule fois dans la chaîne **res**.*

Exemple :

Pour **ch1** = "informatique" et **ch2** = "multimedia", on obtient **res** = "imatue"

L'interface graphique à concevoir contient les éléments suivants, comme l'illustre la capture d'écran ci-dessous :

- Un label contenant le texte "**Intersection de deux chaînes**"
- Un label contenant le texte "**ch1** : "
- Une zone de saisie pour la saisie de **ch1**
- Un label contenant le texte "**ch2** : "
- Une zone de saisie pour la saisie de **ch2**
- Un label pour afficher le résultat
- Un bouton intitulé "**Former**"

Travail demandé :

- 1) Concevoir l'interface graphique présentée précédemment et l'enregistrer sous le nom **InterfaceIntersection**
- 2) Créer un programme Python et l'enregistrer sous le nom **Intersection**, dans lequel, il est demandé :
 - a) de développer une fonction nommée **Recherche (ch1, ch2)** qui permet de former la chaîne **res**.
 - b) de développer un module **Play** qui s'exécute suite à un clic sur le bouton "**Former**" permettant :
 - de récupérer les deux chaînes **ch1** et **ch2** saisies. Les chaînes **ch1** et **ch2** doivent être non vides, de longueurs inférieures à 30 chacune et contiennent seulement des lettres alphabétiques en minuscule.
 - d'exploiter la fonction **Recherche (ch1, ch2)** afin d'afficher le résultat formé via le **label** dédié à l'affichage dans l'interface graphique **InterfaceIntersection**.
 - c) d'ajouter les instructions permettant d'exploiter l'interface graphique intitulée **InterfaceIntersection** en se référant à l'annexe ci-après.

N.B. : l’affichage doit être conforme aux exemples d’executions suivants :

Exemples d’executions :

Intersection de deux chaînes

ch1 :

ch2 :

Veuillez introduire deux chaines non vides

Former

Intersection de deux chaînes

ch1 :

ch2 :

Veuillez introduire deux chaines valides

Former

Intersection de deux chaînes

ch1 :

ch2 :

L'intersection est: imatue

Former

Annexe

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi ("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect (Nom_Module)
app.exec_()
```

Grille d’évaluation

Tâches	Nombre de points
Conception de l’interface InterfaceIntersection	4 pts
Création et enregistrement du programme Intersection	1 pt
Développement de la fonction Recherche	6 pts
Développement du module Play	4 pts
Ajout des instructions de l’exploitation de l’interface	3 pts
Modularité et cohérence	2 pts

```

from PyQt5.QtWidgets import *
from PyQt5.uic import *
def recherche(ch1,ch2):
    if len(ch1)<len(ch2):
        aux=ch1
        ch1=ch2
        ch2=aux
    res=""
    for i in range(len(ch1)):
        if ch2.find(ch1[i])!=-1 and res.find(ch1[i])==-1:
            res=res+ch1[i]
    return res
def verif(ch):
    test=True
    i=0
    while test and i<len(ch):
        if "a"<=ch[i]<="z":
            i=i+1
        else:
            test=False
    return test
def play():
    ch1=f.t1.text()
    ch2=f.t2.text()
    if ch1=="" or ch2=="":
        msg="Veuillez introduire deux chaines non vides "
    elif not(len(ch1)<=30) or not(len(ch2)<=30) or not(verif(ch1)) or not(verif(ch2)):
        msg="Veuillez introduire deux chaines valides"
    else:
        msg="l'intersection est : "+recherche(ch1,ch2)
    f.l.setText(msg)
app=QApplication([])
f=loadUi("InterfaceIntersection.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()

```

RÉPUBLIQUE TUNISIENNE *** Ministère de l'Éducation	EXAMEN DU BACCALAURÉAT	SESSION 2024
	ÉPREUVE PRATIQUE D'INFORMATIQUE	
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques	
	Coefficient de l'épreuve : 0.5	Durée: 1h

Important :

Le candidat est appelé à créer, dans le répertoire **Bac2024**, un dossier de travail ayant comme nom son numéro d'inscription (6 chiffres) et dans lequel il doit enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Divisibilité par 7 (Méthode de Horner)

On se propose de concevoir une interface graphique permettant de saisir un nombre **X** formé de 5 à 20 chiffres, puis de vérifier s'il est divisible par 7 en utilisant le principe suivant :

Etape1 : Former, à partir du nombre **X**, un nouveau nombre **Y** en faisant correspondre à chaque chiffre de **X** le reste de sa division euclidienne par 7.

Exemple

X =	5	5	2	7	5	7	9	8	2
	↓	↓	↓	↓	↓	↓	↓	↓	↓
Y =	5	5	2	0	5	0	2	1	2

Etape2 : Former, à partir du nombre **Y**, un nouveau nombre **Z** en faisant correspondre à chaque tranche de deux chiffres de **Y** (en commençant à partir de la droite), le reste de sa division euclidienne par 7.

Exemple

Y =	5	5	2	0	5	0	2	1	2
	↓	↓	↓	↓	↓	↓	↓	↓	↓
Z =	5	3	5	2	5				

- En effet, pour la tranche 12 de **Y**, lui correspond dans **Z** la valeur 5 (reste de la division euclidienne de 12 par 7).

- De même pour les autres tranches.

- La dernière tranche se compose d'un seul chiffre car le nombre de chiffres de **Y** est impair.

Etape3 : Appliquer au nombre **Z**, la méthode de **Horner** définie par l'algorithme suivant de la fonction **Horner(Z)**.

Fonction Horner(Z : Chaîne de caractères) : Entier

DEBUT

$M \leftarrow 0$

Tant que $Z \neq ""$ Faire

$CH \leftarrow Z[0]$

$M \leftarrow (M * 2 + \text{Valeur}(CH)) \text{ Mod } 7$

$Z \leftarrow \text{Sous_chaîne}(Z, 1, \text{Long}(Z))$

Fin Tant que

Retourner M

FIN

T.D.O.L

Objet	Type/Nature
M	Entier
CH	Chaîne de caractères

Le nombre **X** est divisible par 7 lorsque la fonction **Horner**, appliquée au nombre **Z**, retourne la valeur **zéro**.

Travail demandé

- Créer l'interface graphique illustrée dans la figure **Fig1** et l'enregistrer sous le nom **InterfaceHorner**. Cette interface contient les éléments suivants :
 - Un label contenant le texte "**Divisibilité par 7 (Méthode de Horner)**",
 - Un label contenant le texte "**X=**",
 - Une zone de saisie pour la saisie du nombre **X**,
 - Un label pour afficher un message,
 - Un bouton intitulé "**Vérifier**".

Divisibilité par 7 (Méthode de Horner)

X=

Vérifier

Fig1

2. Créer un programme en Python et l'enregistrer sous le nom **DivHorner** dans lequel on demande :
- d'implémenter l'algorithme de la fonction **Horner**.
 - de développer une fonction **Etape1(X)** qui simule l'Etape1 pour retourner le nombre **Y**.
 - de développer une fonction **Etape2(Y)** qui simule l'Etape2 pour retourner le nombre **Z**.
 - de développer un module **Play** qui s'exécute suite à un clic sur le bouton "**Vérifier**" permettant :
 - de récupérer la valeur du nombre **X** saisi et de s'assurer de sa validité afin d'afficher le message adéquat via le **label** dédié à l'affichage, comme illustré dans la figure **Fig2**.
 - d'exploiter les fonctions **Etape1**, **Etape2** et **Horner** afin d'afficher le message adéquat via le **label** dédié à l'affichage, comme illustré dans les figures **Fig3** et **Fig4**.
 - d'exploiter l'annexe présentée ci-après tout en apportant les modifications nécessaires à l'intégration de l'interface graphique **InterfaceHorner**.

Divisibilité par 7 (Méthode de Horner)

X=

Vérifier

Veuillez saisir un nombre de 5 à 20 chiffres

Fig2

Divisibilité par 7 (Méthode de Horner)

X=

Vérifier

967445987 n'est pas divisible par 7

Fig3

Divisibilité par 7 (Méthode de Horner)

X=

Vérifier

552757982 est divisible par 7

Fig4

Annexe

```

from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi ("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect (Nom_Module)
app.exec_()

```

Grille d'évaluation

Tâches	Nombre de points
1. Création de l'interface InterfaceHorner .	3
2. Création du programme DivHorner .	17 =
a. Implémentation de la fonction Horner .	3
b. Développement de la fonction Etape1 .	3.5
c. Développement de la fonction Etape2 .	4.5
d. Développement du module Play .	4.5
e. Exploitation de l'annexe.	1.5

Bac 2024 8h Divisible par 7 (Méthode de Horner)

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication

def etape1(x):
    y=""
    for i in range(len(x)):
        y=y+str(int(x[i])%7)
    return y
def etape2(y):
    z=""
    if len(y)%2==1:
        y="0"+y
    for i in range(0,len(y),2):
        z=z+str(int(y[i:i+2])%7)
    return z
def horner(z):
    m=0
    while(z!=""):
        ch=z[0]
        m=(m*2+int(ch))%7
        z=z[1:len(z)]
    return m
def play():
    ch=f.t.text()
    if not(ch.isdecimal())or not(5<=len(ch)<=20):
        msg="veuillez saisir un nombre de 5 a 20 chiffres"
    elif horner(etape2(etape1(ch)))==0:
        msg=ch+" est divisible par 7"
    else:
        msg=ch+" n'est pas divisible par 7"
    f.l.setText(msg)
app=QApplication([])
f=loadUi("InterfaceHorner.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()
```

RÉPUBLIQUE TUNISIENNE *** Ministère de l'Éducation	EXAMEN DU BACCALAURÉAT	SESSION 2024
	ÉPREUVE PRATIQUE D'INFORMATIQUE	
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques	
	Coefficient de l'épreuve : 0.5	Durée: 1h

Important :

Le candidat est appelé à créer, dans le répertoire **Bac2024**, un dossier de travail ayant comme nom son numéro d'inscription (6 chiffres) et dans lequel il doit enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Autonombres

On se propose de concevoir une interface graphique permettant de saisir deux entiers **N** ($20 \leq N \leq 50$) et **M** ($N < M \leq 100$), puis d'afficher tous les nombres compris entre **N** et **M** qui sont **autonombres**.

Un **autonombre** **X** est un entier positif qui ne peut pas s'écrire sous la forme d'un entier **Y** ajouté à la somme des chiffres de **Y**.

Exemples :

- **24** n'est pas un **autonombre** puisqu'il est égal à l'entier **21** ajouté à la somme de ses chiffres ($24 = 21 + (2 + 1)$).
- **53** est un **autonombre** puisqu'il ne peut pas s'écrire à partir d'un autre nombre ajouté à la somme de ses chiffres.

Travail demandé

- Créer l'interface graphique illustrée dans la figure **Fig1** et l'enregistrer sous le nom **InterfaceAutonombre**. Cette interface contient les éléments suivants :

- Un label contenant le texte "**Autonombres**",
- Un label contenant le texte "**N=**",
- Une zone de saisie pour la saisie d'un entier **N**,
- Un label contenant le texte "**M=**",
- Une zone de saisie pour la saisie d'un entier **M**,
- Un label pour afficher un message,
- Un bouton intitulé "**Afficher**".

Fig1

- Créer un programme en Python et l'enregistrer sous le nom **Autonombre** dans lequel on demande :
 - d'implémenter l'algorithme suivant de la fonction **SommeChiffres(X)** qui retourne la somme des chiffres d'un entier positif **X**.

Fonction SommeChiffres(X : Entier) : Entier DEBUT $S \leftarrow 0$ Tant que ($X \neq 0$) Faire $S \leftarrow S + X \text{ Mod } 10$ $X \leftarrow X \text{ Div } 10$ Fin Tant que Retourner S FIN	
T.D.O.L	
Objet	Type/Nature
S	Entier

- b. de développer une fonction booléenne nommée **Vérifier(X)**, qui exploite la fonction **SommeChiffres**, pour vérifier si l'entier **X** est un **autonombre**.
- c. de développer une fonction nommée **Chercher(N, M)**, qui exploite la fonction **Vérifier**, pour former une chaîne de caractères contenant tous les nombres **autonombres** compris entre **N** et **M** séparés par le caractère "-" (voir **Fig4**).
- d. de développer un module **Play** qui s'exécute suite à un clic sur le bouton "**Afficher**" permettant :
 - de récupérer les valeurs des entiers **N** et **M** saisis et de s'assurer de leurs validités afin d'afficher le message adéquat via le **label** dédié à l'affichage, comme illustré dans la figure **Fig2**.
 - d'exploiter la fonction **Chercher** pour afficher le message adéquat via le **label** dédié à l'affichage, comme illustré dans les figures **Fig3** et **Fig4**.
- e. d'exploiter l'annexe présentée ci-après tout en apportant les modifications nécessaires à l'intégration de l'interface graphique **InterfaceAutonombre**.

Autonombres

N= M=

Afficher

Veillez respecter : $20 \leq N \leq 50$ et $N < M \leq 100$

Fig2

Autonombres

N= M=

Afficher

Aucun Autonombre entre 35 et 40

Fig3

Autonombres

N= M=

Afficher

Le(s) nombre(s) Autonombre(s) : 20-31-42-53-64-75-86-97

Fig4

Annexe

```

from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi ("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect (Nom_Module)
app.exec_()

```

Grille d'évaluation

Tâches	Nombre de points
1. Création de l'interface InterfaceAutonombre .	3
2. Création du programme Autonombre .	17 =
a. Implémentation de la fonction SommeChiffres .	3
b. Développement de la fonction Vérifier .	4.5
c. Développement de la fonction Chercher .	4
d. Développement du module Play .	4
e. Exploitation de l'annexe.	1.5

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
def sommechiffres(x):
    s=0
    while(x!=0):
        s=s+x%10
        x=x//10
    return s
def verifier(x):
    i=x//2
    test=True
    while test and (i<=x-1):
        if sommechiffres(i)+i==x:
            test=False
        else:
            i=i+1
    return test
def chercher(n,m):
    ch=""
    for i in range(n,m+1):
        if verifier(i):
            ch=ch+str(i)+'-'
    if ch!="":
        ch=ch[:len(ch)-1]
    return ch
def play():
    ch1=f.t1.text()
    ch2=f.t2.text()
    if not(ch1.isdecimal()) or not(ch2.isdecimal()) or not(20<=int(ch1)<=50) or not(int(ch1)<=int(ch2)<=200):
        msg="veuillez respecter 20<=N<=50 et N<=M<=100"
    elif chercher(int(ch1),int(ch2))=="":
        msg="aucun nombre Autonombre entre "+ch1+" "+ch2
    else:
        msg="les nombres Autonombre entre "+ch1+" et "+ch2+" = "+chercher(int(ch1),int(ch2))
    f.l.setText(msg)
app=QApplication([])
f=loadUi("InterfaceAutonombre.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()
```

RÉPUBLIQUE TUNISIENNE *** Ministère de l'Éducation	EXAMEN DU BACCALAURÉAT	SESSION 2024
	ÉPREUVE PRATIQUE D'INFORMATIQUE	
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques	
	Coefficient de l'épreuve : 0.5	Durée: 1h

Important :

Le candidat est appelé à créer, dans le répertoire **Bac2024**, un dossier de travail ayant comme nom son numéro d'inscription (6 chiffres) et dans lequel il doit enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Transformation Powertrain

On se propose de concevoir une interface graphique permettant de saisir deux entiers **N** ($200 \leq N \leq 999999$) et **M** ($3 \leq M \leq 10$), puis d'afficher la **transformation Powertrain** de l'entier **N** ainsi que celles des **M** entiers consécutifs qui le suivent.

La **transformation Powertrain** d'un entier **X** consiste à générer un entier **Y** à partir des chiffres de **X**, en multipliant dans l'ordre chaque chiffre de rang impair élevé à la puissance du chiffre de rang pair.

Il est à noter que :

- Si le nombre de chiffres de **X** est impair alors le dernier chiffre est élevé à la puissance 1.
- Si un chiffre de rang impair est égal à **zéro** et il est suivi d'un **zéro** alors la puissance 0^0 sera remplacée par 1.

Exemples :

- Pour **X = 523**, le nombre généré est $Y = 5^2 \times 3^1 = 25 \times 3 = 75$
- Pour **X = 160071**, le nombre généré est $Y = 1^6 \times 0^0 \times 7^1 = 1 \times 1 \times 7 = 7$

Travail demandé

- Créer l'interface graphique illustrée dans la figure **Fig1** et l'enregistrer sous le nom **InterfacePowertrain**. Cette interface contient les éléments suivants :

- Un label contenant le texte "**Transformation Powertrain**",
- Un label contenant le texte "**N=**",
- Une zone de saisie pour la saisie d'un entier **N**,
- Un label contenant le texte "**M=**",
- Une zone de saisie pour la saisie d'un entier **M**,
- Un label pour afficher un premier message,
- Un label pour afficher un deuxième message,
- Un bouton intitulé "**Transformer**".

Fig 1

- Créer un programme en Python et l'enregistrer sous le nom **Powertrain** dans lequel on demande :
 - d'implémenter l'algorithme suivant de la fonction **Puissance(A, B)** qui retourne la valeur d'un entier **A** élevé à la puissance **B** (notée A^B avec **A** et **B** deux entiers positifs).

Fonction Puissance(A, B : Entier) : Entier

DEBUT

$P \leftarrow 1$

Pour **K** de 1 à **B** Faire

$P \leftarrow P * A$

Fin Pour

Retourner **P**

FIN

T.D.O.L

Objet	Type/Nature
P, K	Entier

- b. de développer une fonction nommée **Transformer(X)** qui exploite la fonction **Puissance** pour déterminer la **transformation Powertrain** de l'entier **X**.
- c. de développer une fonction nommée **Chercher(N, M)**, qui exploite la fonction **Transformer** pour former une chaîne de caractères contenant la transformation **Powertrain** de l'entier **N** ainsi que celles des **M** entiers consécutifs qui le suivent séparés par le caractère "-" (voir **Fig3**).
- d. de développer un module **Play**, qui s'exécute suite à un clic sur le bouton "**Transformer**" permettant :
 - de récupérer les valeurs des deux entiers saisis **N** et **M** et de s'assurer de leurs validités afin d'afficher le message adéquat via le **label** dédié à l'affichage, comme illustré dans la figure **Fig2**,
 - d'exploiter la fonction **Chercher** afin d'afficher les messages adéquats via les **labels** dédiés à l'affichage, comme illustré dans la figure **Fig3**.
- e. d'exploiter l'annexe présentée ci-dessous tout en apportant les modifications nécessaires à l'intégration de l'interface graphique **InterfacePowertrain**.

Transformation Powertrain

N= **M=**

Transformer

Veuillez respecter $200 \leq N \leq 999999$ et $3 \leq M \leq 10$

Fig 2

Transformation Powertrain

N= **M=**

Transformer

La transformation de 523 et les 5 entiers suivants sont :

75-100-125-150-175-200

Fig 3

Annexe

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi ("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect (Nom_Module)
app.exec_()
```

Grille d'évaluation

Tâches	Nombre de points
1. Création de l'interface InterfacePowertrain .	3
2. Création du programme Powertrain .	17 =
a. Implémentation de la fonction Puissance .	3
b. Développement de la fonction Transformer .	5.5
c. Développement de la fonction Chercher .	3.5
d. Développement du module Play .	3.5
e. Exploitation de l'annexe.	1.5

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
def puissance(a,b):
    p=1
    for i in range(1,b+1):
        p=p*a
    return p
def transformer(x):
    ch=str(x)
    y=1
    if len(ch)%2==1:
        ch=ch+"1"
    for i in range(0,len(ch),2):
        y=y*puissance(int(ch[i]),int(ch[i+1]))
    return y
def chercher(n,m):
    ch=""
    for i in range(n,n+m+1):
        ch=ch+str(transformer(i))+"-"
    return ch[:len(ch)-1]
def play():
    ch1=f.t1.text()
    ch2=f.t2.text()
    if not(ch1.isdecimal()) or not(ch2.isdecimal()) or not(200<=int(ch1)<=999999) or not(3<=int(ch2)<=10):
        msg="veuillez respecter 200<=N<=999999 et 3<=M<=10"
        f.l.setText(msg)
    else:
        msg="La transformation de "+ch1+" et les "+ch2+" entiers suivants sont :"
        f.l.setText(msg)
        msg2=chercher(int(ch1),int(ch2))
        f.l2.setText(msg2)
app=QApplication([])
f=loadUi("InterfacePowertrain.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()
```

RÉPUBLIQUE TUNISIENNE *** Ministère de l'Éducation	EXAMEN DU BACCALAURÉAT	SESSION 2024
	ÉPREUVE PRATIQUE D'INFORMATIQUE	
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques	
	Coefficient de l'épreuve : 0.5	Durée: 1h

Important :

Le candidat est appelé à créer, dans le répertoire **Bac2024**, un dossier de travail ayant comme nom son numéro d'inscription (6 chiffres) et dans lequel il doit enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Nombres Premiers

On se propose de concevoir une interface graphique permettant de saisir deux entiers **N** ($10 \leq N \leq 50$) et **M** ($N < M \leq 200$), puis d'afficher tous les nombres **primaires** compris entre **N** et **M**.

Un nombre **P** est dit **primaire** s'il existe un nombre premier **X** tel que $P = X^k$ avec **k** un entier strictement positif.

Un nombre **premier** est un entier strictement positif qui admet exactement deux diviseurs positifs distincts 1 et lui-même.

Exemples :

- 144 n'est pas un nombre **primaire**. En effet, $144 = 12^2$ et 12 n'est pas un nombre premier.
- 169 est un nombre **primaire**. En effet, $169 = 13^2$ et 13 est un nombre premier.

Travail demandé

- Créer l'interface graphique illustrée dans la figure **Fig1** et l'enregistrer sous le nom **InterfacePrimaire**. Cette interface contient les éléments suivants :

- Un label contenant le texte "**Nombres Premiers**",
- Un label contenant le texte "**N=**",
- Une zone de saisie pour la saisie d'un entier **N**,
- Un label contenant le texte "**M=**",
- Une zone de saisie pour la saisie d'un entier **M**,
- Un label pour afficher un message,
- Un bouton intitulé "**Afficher**".

Fig1

- Créer un programme en Python et l'enregistrer sous le nom **Primaire** dans lequel on demande :
 - d'implémenter l'algorithme suivant de la fonction **Premier(X)** qui vérifie si un entier strictement positif **X** est premier.

Fonction Premier (*X : Entier*) : Booléen

DEBUT

TEST $\leftarrow$ Vrai

Si $X = 1$ Alors TEST $\leftarrow$ Faux

Sinon Si ($X > 3$) Alors

K $\leftarrow$ 2

Tant que TEST Et ($K \leq X \text{ Div } 2$) Faire

Si $X \text{ Mod } K = 0$ alors TEST $\leftarrow$ Faux

Sinon $K = K + 1$

Fin Tant que

Retourner TEST

FIN

T.D.O.L

Objet	Type/Nature
K	Entier
TEST	Booléen

- b. de développer une fonction booléenne nommée **Vérifier(P)**, qui exploite la fonction **Premier**, afin de vérifier si l'entier **P** est un nombre **primaire**.
- c. de développer une fonction nommée **Chercher(N, M)**, qui exploite la fonction **Vérifier**, pour former une chaîne de caractères contenant tous les nombres **primaires** compris entre **N** et **M** séparés par le caractère "-" (voir **Fig4**).
- d. de développer un module **Play** qui s'exécute suite à un clic sur le bouton "**Afficher**" permettant :
 - de récupérer les valeurs des entiers **N** et **M** saisis et de s'assurer de leurs validités afin d'afficher le message adéquat via le **label** dédié à l'affichage, comme illustré dans la figure **Fig2**.
 - d'exploiter la fonction **Chercher** pour afficher le message adéquat via le **label** dédié à l'affichage, comme illustré dans les figures **Fig3** et **Fig4**.
- e. d'exploiter l'annexe présentée ci-après tout en apportant les modifications nécessaires à l'intégration de l'interface graphique **InterfacePrimaire**.

Nombres Primaires

N= M=

Afficher

Veuillez respecter : $10 \leq N \leq 50$ et $N < M \leq 200$

Fig2

Nombres Primaires

N= M=

Afficher

Aucun nombre primaire entre 35 et 45

Fig3

Nombres Primaires

N= M=

Afficher

Nombre(s) primaire(s) : 16-25-27-32-49-64-81-121-125-128-169

Fig4

Annexe

```

from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi ("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect (Nom_Module)
app.exec_()

```

Grille d'évaluation

Tâches	Nombre de points
1. Création de l'interface InterfacePrimaire .	3
2. Création du programme Primaire .	17 =
a. Implémentation de la fonction Premier .	3
b. Développement de la fonction Vérifier .	4.5
c. Développement de la fonction Chercher .	4
d. Développement du module Play .	4
e. Exploitation de l'annexe.	1.5

Bac 2024 13h PRIMAIRE

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
from math import *
def premier(n):
    test=True
    i=2
    while test and i<=n//2:
        if n%i==0:
            test=False
        else:
            i=i+1
    return test
def primaire(p):
    test=False
    x=2
    while test==False and x<=p//2:
        k=2
        while test==False and k<=p//2:
            if x**k==p:
                test=True
                #print(x,k)
                #print(x,'|',k)
                k=k+1
            x=x+1
            #print(x)
    return test

def chercher(n,m):
    ch=""
    for i in range(n,m+1):
        if primaire(i):
            ch=ch+str(i)+'-'
    if ch!="":
        ch=ch[:len(ch)-1]
    return ch
```

```
def play():
    ch1=f.t1.text()
    ch2=f.t2.text()
    if not(ch1.isdecimal()) or not(ch2.isdecimal()) or
    not(10<=int(ch1)<=50) or
    not(int(ch1)<=int(ch2)<=200):
        msg="veuillez respecter 10<=N<=50 et
N<=M<=200"
    elif chercher(int(ch1),int(ch2))=="":
        msg="aucun nombre primaire entre "+ch1+"
"+ch2
    else:
        msg="les nombres primaires entre "+ch1+" et
"+ch2+" = "+chercher(int(ch1),int(ch2))
    f.l.setText(msg)
    app=QApplication([])
    f=loadUi("InterfacePrimaire.ui")
    f.show()
    f.b.clicked.connect(play)
    app.exec_()
```

RÉPUBLIQUE TUNISIENNE *** Ministère de l'Éducation	EXAMEN DU BACCALAURÉAT	SESSION 2024
	ÉPREUVE PRATIQUE D'INFORMATIQUE	
	Sections : Mathématiques, Sciences expérimentales et Sciences techniques	
	Coefficient de l'épreuve : 0.5	Durée: 1h

Important :

Le candidat est appelé à créer, dans le répertoire **Bac2024**, un dossier de travail ayant comme nom son numéro d'inscription (6 chiffres) et dans lequel il doit enregistrer, au fur et à mesure, tous les fichiers solution à ce sujet.

Suite arithmétique

On se propose de concevoir une interface graphique permettant de saisir un nombre **X**, de longueur paire et composé de 6 à 20 chiffres, puis de vérifier si toutes les tranches successives de deux chiffres de **X**, une fois triées dans l'ordre croissant, constituent des termes d'une suite arithmétique croissante.

Une **suite arithmétique** croissante est une suite dans laquelle chaque terme permet de déduire le suivant en lui ajoutant une constante **r** appelée raison (pour tout $n > 0$; $U_n = U_{n-1} + r$ avec $r > 0$).

Exemples :

- Pour **X = 301015250520** ; Les tranches de X : 3 0 1 0 1 5 2 5 0 5 2 0
Les tranches de X triées : 0 5 1 0 1 5 2 0 2 5 3 0

Les tranches successives de **X**, une fois triées dans l'ordre croissant, forment des termes d'une suite arithmétique croissante de raison $r = 5$. En effet, $(10 - 5) = (15 - 10) = (20 - 15) = (25 - 20) = (30 - 25) = 5$

- Pour **X = 41331020** ; Les tranches de X : 4 1 3 3 1 0 2 0
Les tranches de X triées : 1 0 2 0 3 3 4 1

Les tranches successives de **X**, une fois triées dans l'ordre croissant, ne forment pas des termes d'une suite arithmétique croissante.

Travail demandé

1. Créer l'interface graphique illustrée dans la figure **Fig1** et l'enregistrer sous le nom **InterfaceArithmétique**. Cette interface contient les éléments suivants :

- Un label contenant le texte "**Suite arithmétique**"
- Un label contenant le texte "**X=**",
- Une zone de saisie pour la saisie du nombre **X**,
- Un label pour afficher un premier message,
- Un label pour afficher un deuxième message,
- Un bouton intitulé "**Vérifier**".

Fig1

2. Créer un programme en Python et l'enregistrer sous le nom **SuiteArithmétique** dans lequel on demande :

- a. d'implémenter l'algorithme suivant de la fonction **Trier(CH)** qui permet de trier dans l'ordre croissant les tranches de deux chiffres de la chaîne **CH**.

Fonction Trier(CH : Chaîne de caractères) : Chaîne de caractères

DEBUT

PERMUT ← Vrai

Tant que PERMUT Faire

PERMUT ← Faux

Pour K de 0 à Long(CH)-3 [Pas=2] Faire

BLOC1 ← Sous_chaîne(CH, K, K+2)

BLOC2 ← Sous_chaîne(CH, K+2, K+4)

Si BLOC1 > BLOC2 Alors

CH ← Sous_chaîne(CH, 0, K) + BLOC2 + BLOC1 + Sous_chaîne(CH, K+4, Long(CH))

PERMUT ← Vrai

FinSi

Fin Pour

Fin Tant que

Retourner CH

FIN

T.D.O.L

Objet	Type/Nature
K	Entier
PERMUT	Booléen
BLOC1, BLOC2	Chaîne de caractères

- b. de développer une fonction nommée **Calculer(X)** permettant de retourner, pour un nombre **X** dont les tranches de deux chiffres sont triées dans l'ordre croissant, un entier égal à :
- la différence entre deux tranches successives de **X** dans le cas où cette différence est constante pour toutes les tranches successives,
 - **zéro** dans le cas contraire.
- c. de développer un module **Play** qui s'exécute suite à un clic sur le bouton "**Vérifier**" permettant :
- de récupérer la valeur du nombre **X** saisi et de s'assurer de sa validité afin d'afficher le message adéquat via le **label** dédié à l'affichage, comme illustré dans la figure **Fig2**,
 - d'exploiter les fonctions **Trier** et **Calculer** afin d'afficher le message adéquat via les **labels** dédiés à l'affichage, comme illustré dans les figures **Fig3** et **Fig4**.
- d. d'exploiter l'annexe présentée ci-après tout en apportant les modifications nécessaires à l'intégration de l'interface graphique **InterfaceArithmétique**.

Suite arithmétique

X=

Vérifier

Veuillez saisir un nombre de 6 à 20 chiffres et de longueur paire

Fig2

Suite arithmétique

X=

Vérifier

Les tranches de chiffres triées 10203341
ne forment pas des termes d'une suite arithmétique

Fig3

Suite arithmétique

X=

Vérifier

Les tranches de chiffres triées 051015202530
forment des termes d'une suite arithmétique (r=5)

Fig4

Annexe

```

from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
.....
.....
app = QApplication([])
windows = loadUi ("Nom_Interface.ui")
windows.show()
windows.Nom_Bouton.clicked.connect (Nom_Module)
app.exec_()

```

Grille d'évaluation

Tâches	Nombre de points
1. Création de l'interface InterfaceArithmétique .	3
2. Création du programme SuiteArithmétique .	17 =
a. Implémentation de la fonction Trier .	4.5
b. Développement de la fonction Calculer .	6
c. Développement du module Play	5
d. Exploitation de l'annexe.	1.5

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
def trier(ch):
    print(ch)
    permut=True
    while permut:
        permut=False
        for i in range(0,len(ch)-2,2):
            bloc1=ch[i:i+2]
            bloc2=ch[i+2:i+4]
            if(bloc1>bloc2):
                ch=ch[:i]+bloc2+bloc1+ch[i+4:]
                permut=True
    return ch
def calculer(x):
    r=int(x[2:4])-int(x[:2])
    k=2
    constante=True
    while k<len(x)-2 and constante:
        constante=r==int(x[k+2:k+4])-int(x[k:k+2])
        k=k+2
    if not(constante):
        r=0
    return r
def play():
    ch=f.t.text()
    if not(ch.isdecimal())or not(6<=len(ch)<=20)or len(ch)%2!=0:
        msg1="veuillez saisir un nombre de 6 a 20 chiffres et de longueur paire"
        msg2=""
    else:
        msg1="les tranches de chiffres triées "+trier(ch)
        m=calculer(trier(ch))
        if m!=0:
            msg2="forment des termes d'une suite arithmétique(r="+str(m)+")"
        else:
            msg2="ne forment pas des termes d'une suite arithmétique"
    f.l1.setText(msg1)
    f.l2.setText(msg2)
app=QApplication([])
f=loadUi("InterfaceArithmétique.ui")
f.show()
f.b.clicked.connect(play)
app.exec_()
```